



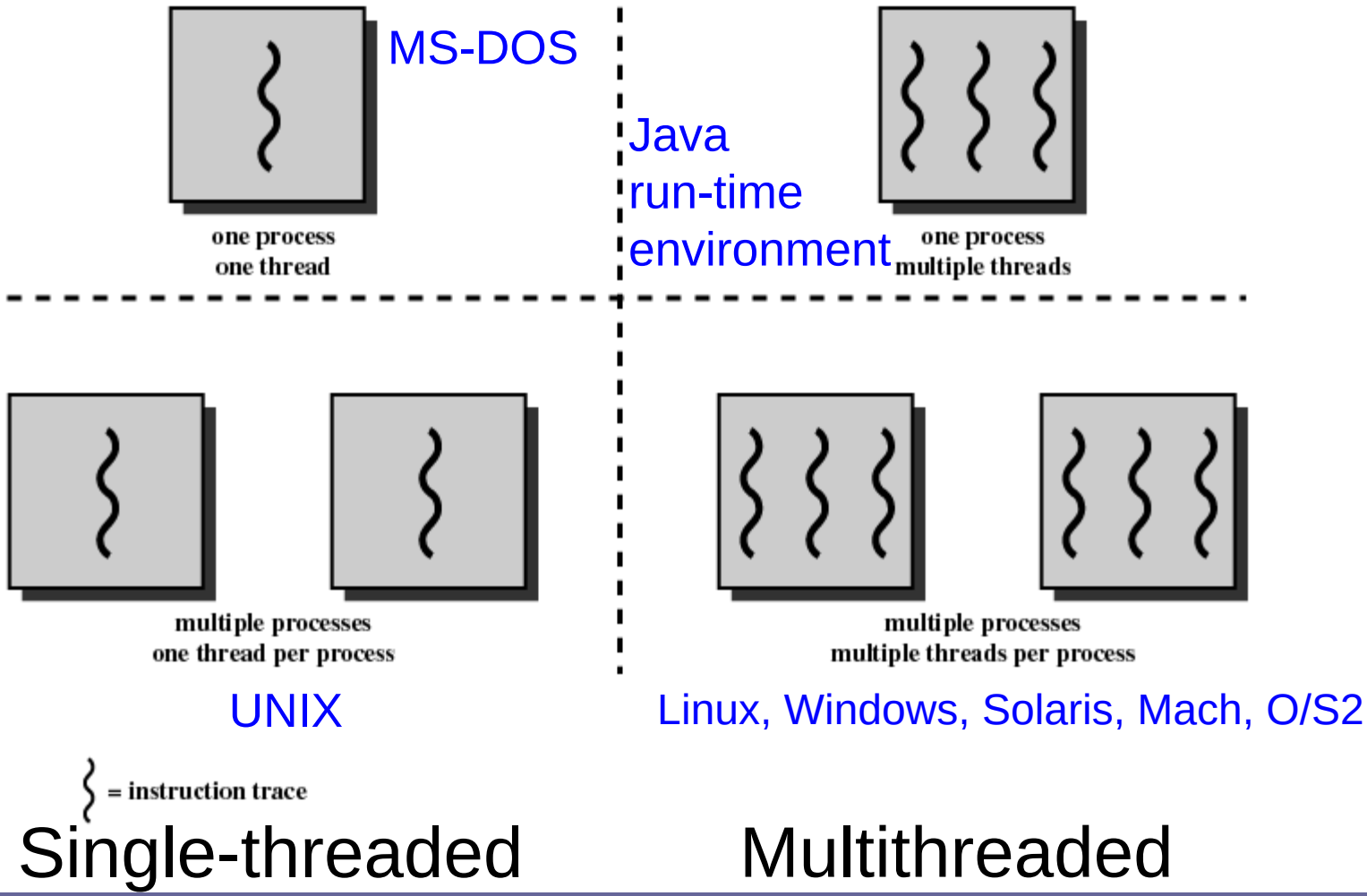
Threads, SMP, and Microkernels

Chapter 4



Multithreading

4 Operating system supports multiple threads of execution within a single process





Multithreading

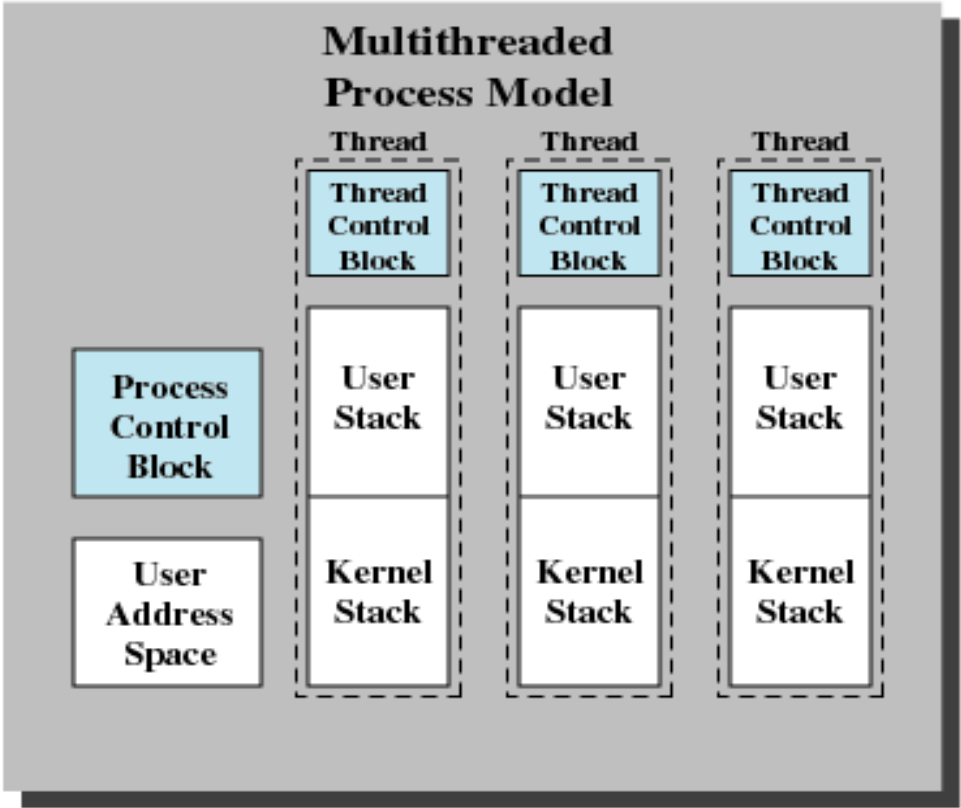
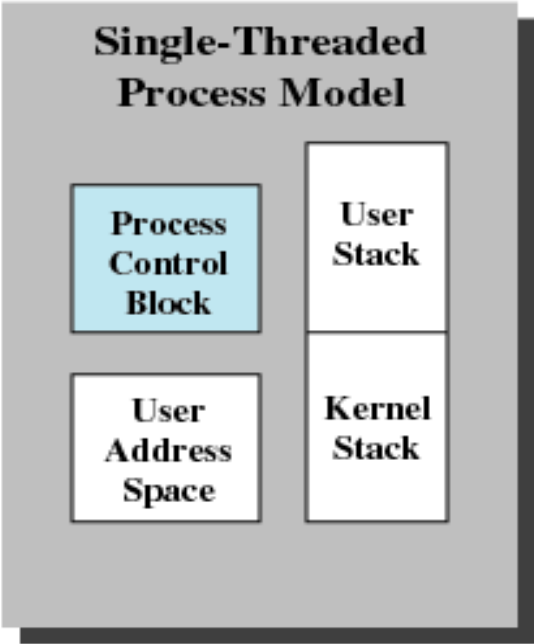
4 Each thread in a process has

- an execution state (running, ready, etc.)
- saved thread context when not running
- an execution stack
- some per-thread static storage for local variables
- access to the memory and resources of its process
 - all threads of a process share this



Multithreading

- ▶ Thread control block containing register values, priority and other state information.





Benefits of Threads

- 4 Takes less time to create a new thread than a new process
- 4 Take less time to terminate a thread
- 4 Take less time to switch between two threads within the same process
- 4 Since threads within the same process share memory and files, they can communicate with each other without invoking the kernel



Drawbacks of Threads

- 4 Requires careful design (shared variables)
- 4 Hard to debug because the interaction between threads very hard to control.



Multithreading

Uses of threads in a single-user multiprocessing system

- 4 Foreground and background work, example: one thread could display menus and read user input, another thread executes user commands
- 4 Asynchronous processing: a thread to backup data periodically
- 4 Speed of execution: on a multiprocessor system, multiple threads can execute simultaneously
- 4 Modular program structure: programs with many activities or input and output can implement using threads



Multithreading

formatting

Four score and seven years ago, our fathers brought forth upon this continent a new nation: conceived in liberty, and dedicated to the proposition that all men are created equal. Now we are engaged in a great civil war testing whether that	nation, or any nation so conceived and so dedicated, can long endure. We are met on a great battlefield of that war. We have come to dedicate a portion of that field as a final resting place for those who here gave their	lives that this nation might live. It is altogether fitting and proper that we should do this. But, in a larger sense, we cannot consecrate we cannot hallow this ground. The brave men, living and dead,	who struggled here have consecrated it, far above our poor power to add or detract. The world will little note, nor long remember, what we say here, but it can never forget what they did here. It is for us the living, rather, to be dedicated	here to the unfinished work which they who fought here have thus far so nobly advanced. It is rather for us to be here dedicated to the great task remaining before us, that from these honored dead we take increased devotion to that cause for which	they gave the last full measure of devotion, that we here highly resolve that these dead shall not have died in vain that this nation, under God, shall have a new birth of freedom and that government of the people by the people, for the people
---	--	---	---	---	---

Spell & grammar checking

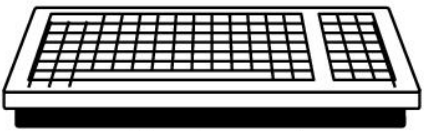


Auto saving

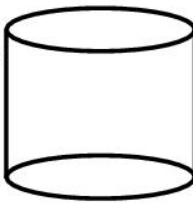
editing

Kernel

Word processor



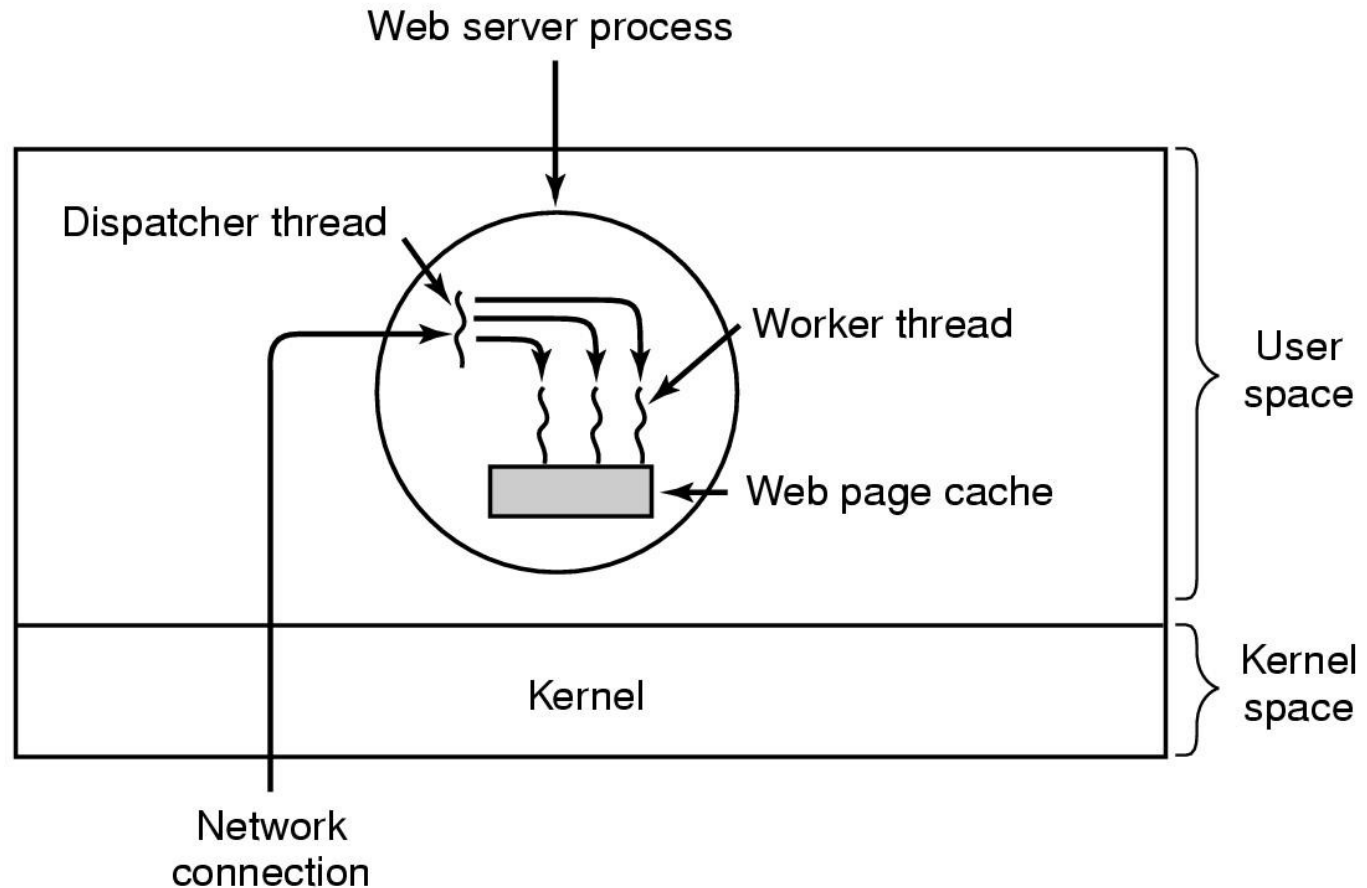
Keyboard



Disk



Multithreading



Dispatcher accepts new request and passes it to an idle worker.
Worker tries to get the page from cache, if not in cache, read from disk (blocked)



Multithreading

- 4 Suspending a process involves suspending all threads of the process since all threads share the same address space
- 4 Termination of a process, terminates all threads within the process



Thread States

4 States associated with a change in thread state

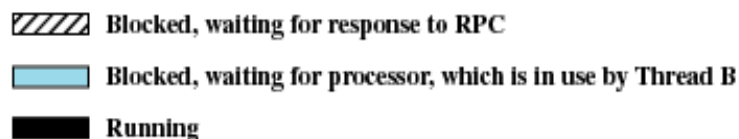
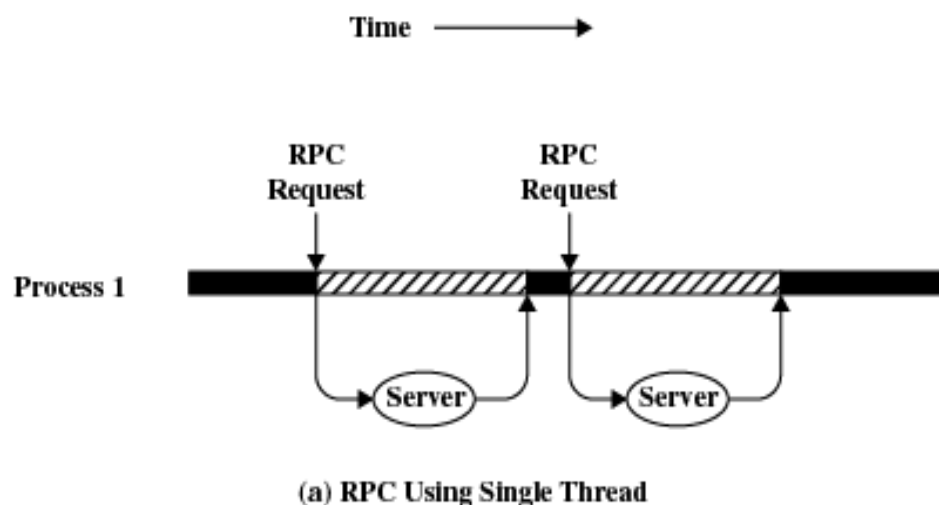
- Spawn
 - A thread within a process may spawn another thread
- Block
- Unblock
- Finish
 - De-allocate register context and stacks

4 Issue: Does blocking a thread results in blocking of the entire process?



Remote Procedure Call (Single Thread)

- 4 An RPC is a technique by which two programs, which may execute on different machines, interact using procedure call/return syntax. RPCs are often used for client/server applications.

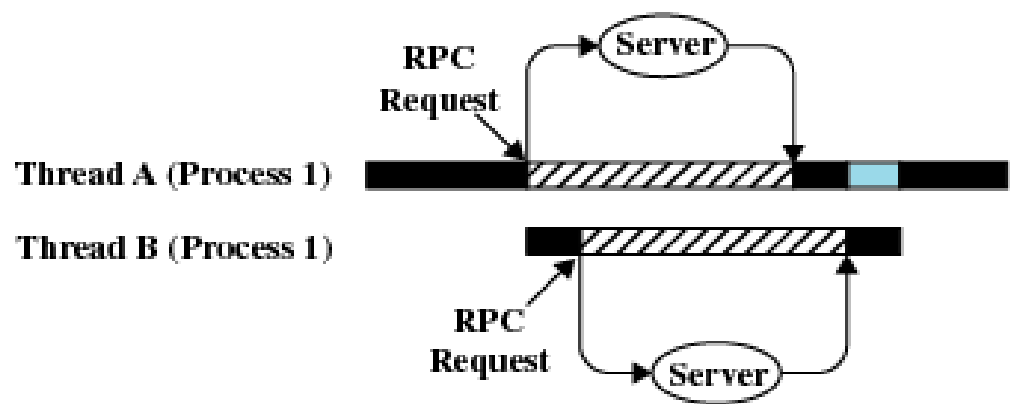


- 4 In a single-threaded program, the results in the above RPCs are obtained in sequence, so that the program has to wait for a response from each server in turn.



Remote Procedure Call Using Threads

- 4 In a dual-threaded program on a uniprocessor, the requests must be generated sequentially, and the results processed in sequence.
- 4 However, the program waits concurrently for the two replies, hence reduced waiting time significantly.



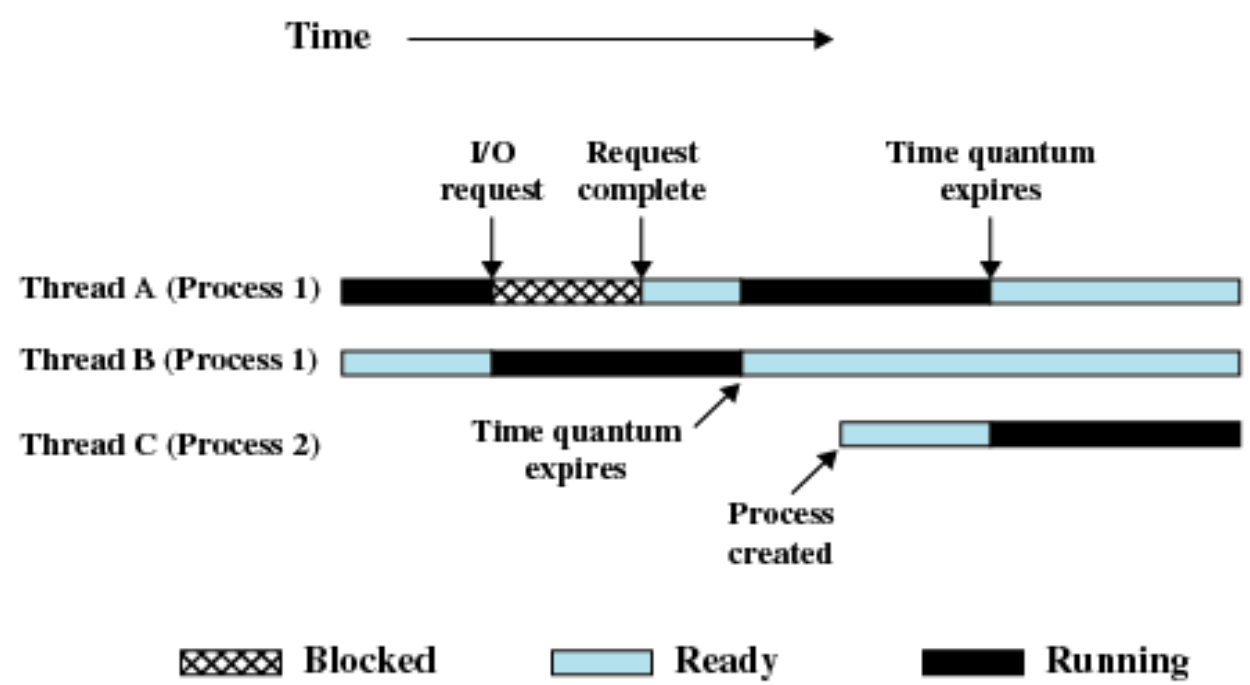
(b) RPC Using One Thread per Server (on a uniprocessor)

- Unblocked waiting for response to RPC
- Blocked, waiting for processor, which is in use by Thread B
- Running



Multiprogramming on Uniprocessor

4 On a uniprocessor, multiprogramming enables the interleaving of multiple threads within multiple processes.





Implementation

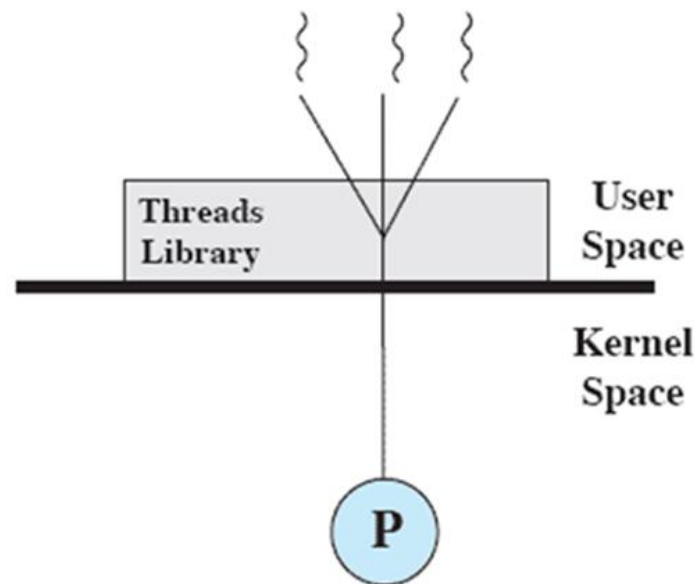
4 2 categories of thread implementation:

- User-level threads (ULT)
- Kernel-level threads (KLT)



User-Level Threads

- ▶ All thread management is done by the application
- ▶ The kernel is not aware of the existence of threads (schedule on process basis & assign single process state)
- ▶ Applications can be programmed to be multithreaded using threads library that contains code for
 - creating and destroying threads
 - passing messages and data between threads
 - scheduling thread execution
 - saving and restoring thread contexts

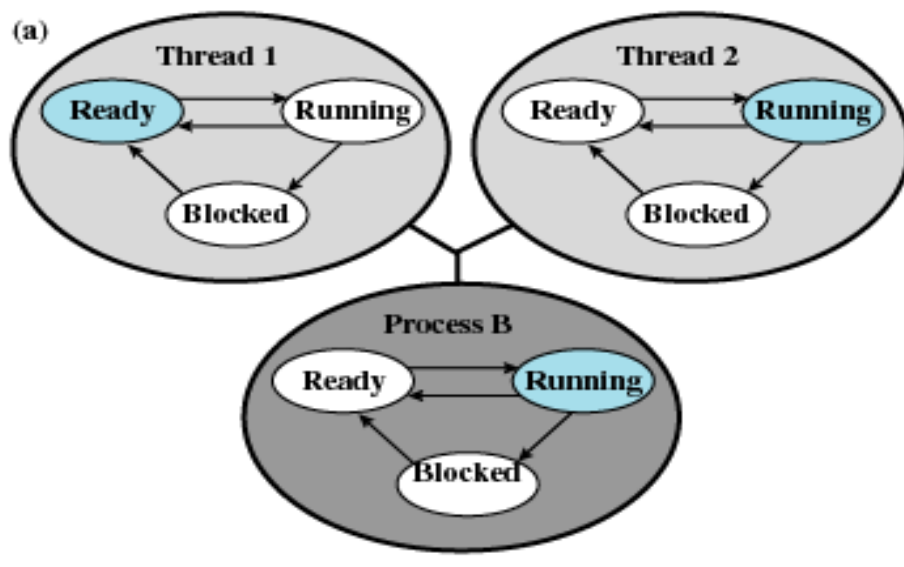


(a) Pure user-level



ULT States vs. Process States

- ▶ Suppose thread 2 in process B is executing



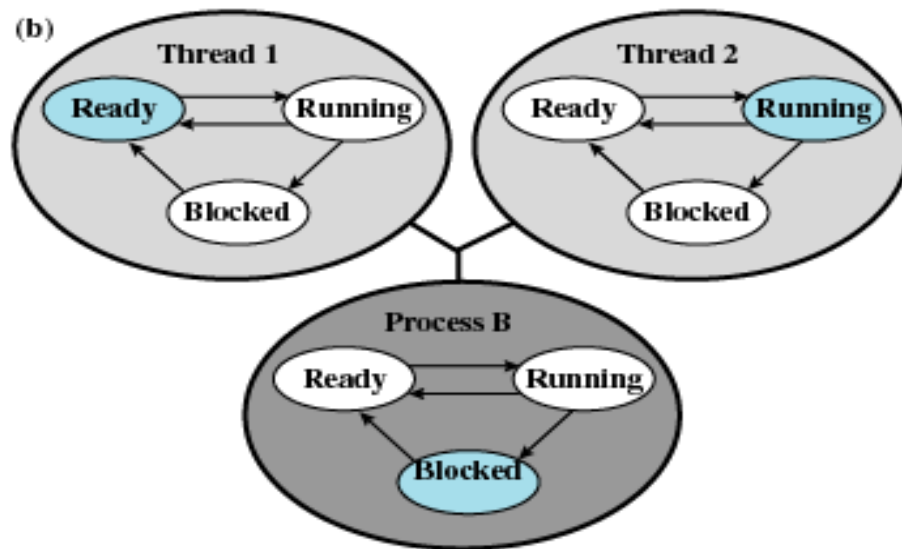
- ▶ 3 possible cases/occurrence:



ULT States vs. Process States

► Possible occurrence 1:

- Application in thread 2 makes a I/O call that blocks process B, the kernel switches to another process. Although thread 2 is perceived in Running state by the thread library, it is not actually executed on a processor.

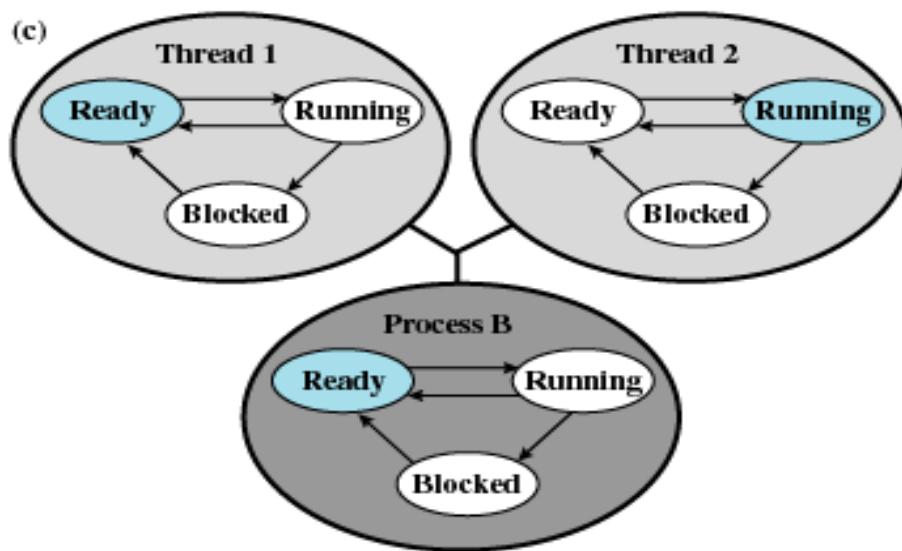




ULT States vs. Process States

► Possible occurrence 2:

- Process B exhausted its time slice and is placed in Ready state, but the data structure managed by thread library shows that thread 2 is still in running state. Again, Thread 2 is not actually executed on a processor.

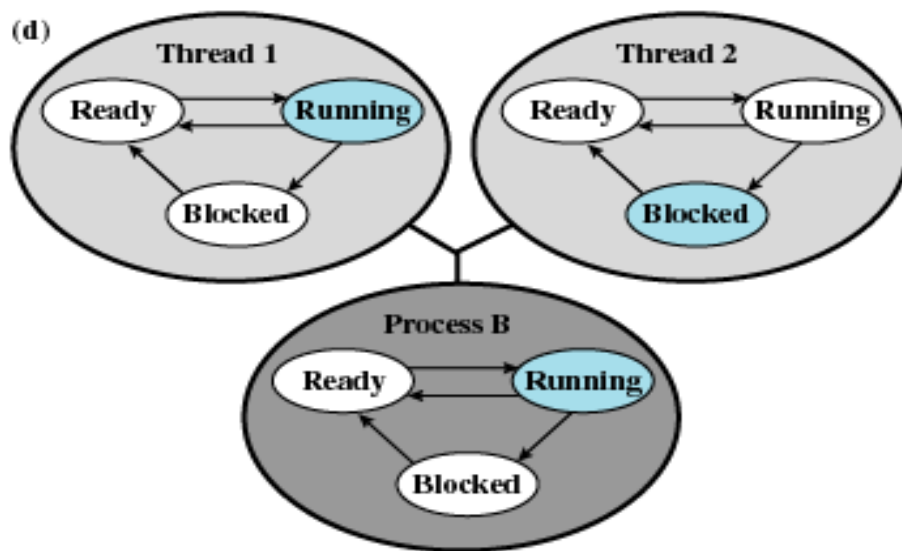




ULT States vs. Process States

► Possible occurrence 3:

- Thread 2 needs some actions performed by thread 1. Thread 2 is blocked and thread 1 is executing. Process B still remains in Running state.





Advantages of ULTs

- ▶ Thread switching does not require kernel mode privileges → save the overhead of mode switches
- ▶ Scheduling algorithm can be tailored to application without disturbing OS scheduler.
- ▶ Can run on any OS. Threads library is a set of application-level utilities.



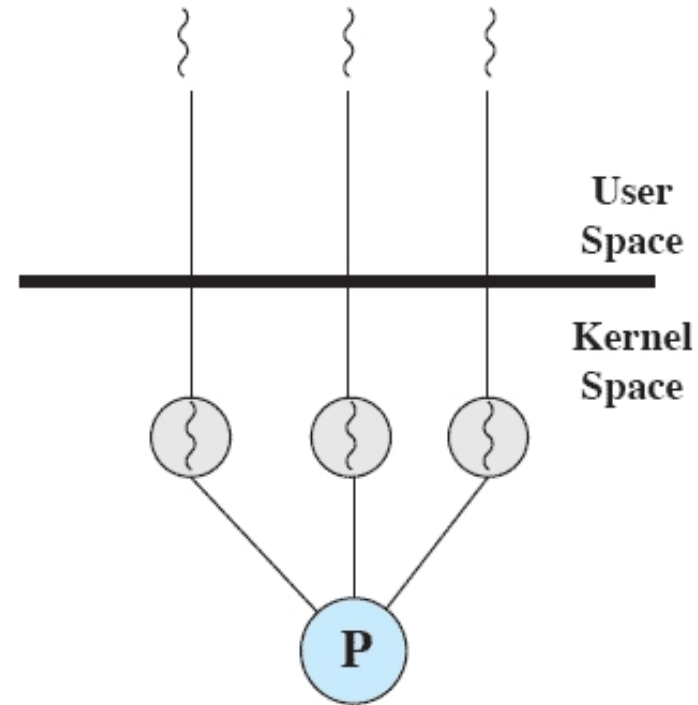
Disadvantages of ULTs

- ▶ When a ULT makes a system call, all of the threads in the process are blocked because the entire process is blocked and taken out of run queue.
- ▶ Multithreaded application cannot take advantage of multiprocessing in a pure ULT strategy → one process is assigned to one processor → only one thread can execute at a time.



Kernel-Level Threads

- 4 Thread management done by the kernel.
- 4 Windows is an example of this approach
- 4 No thread management code in application area, only application programming interface (API) to the kernel thread facility.
- 4 Kernel maintains context information for the process and the threads
- 4 Scheduling is done on a thread basis



(b) Pure kernel-level



Advantages of KLT

- 4 The kernel can simultaneously schedule multiple threads from a process on multiple processors.
- 4 If the thread is blocked, the kernel can schedule another thread from the same process.
- 4 The kernel themselves can be multithreaded.



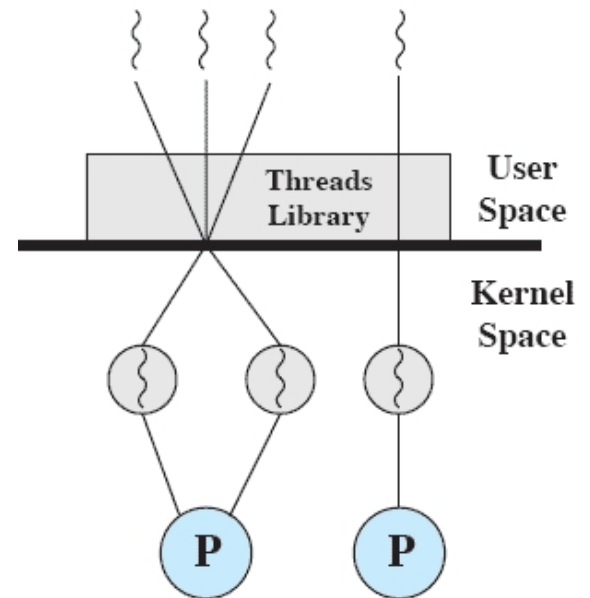
Disadvantage of KLT

- 4 The transfer of control to another thread in the same process requires a mode switch to the kernel. This introduces additional latencies.



Combined Approaches

- 4 Example is Solaris
- 4 Thread creation done in the user space
- 4 Scheduling and synchronization of threads within application
- 4 ULTs from an application are mapped onto KLTs.
- 4 Multiple threads can run concurrently on multiple processors
- 4 A blocking system call need not block the entire process.



(c) Combined



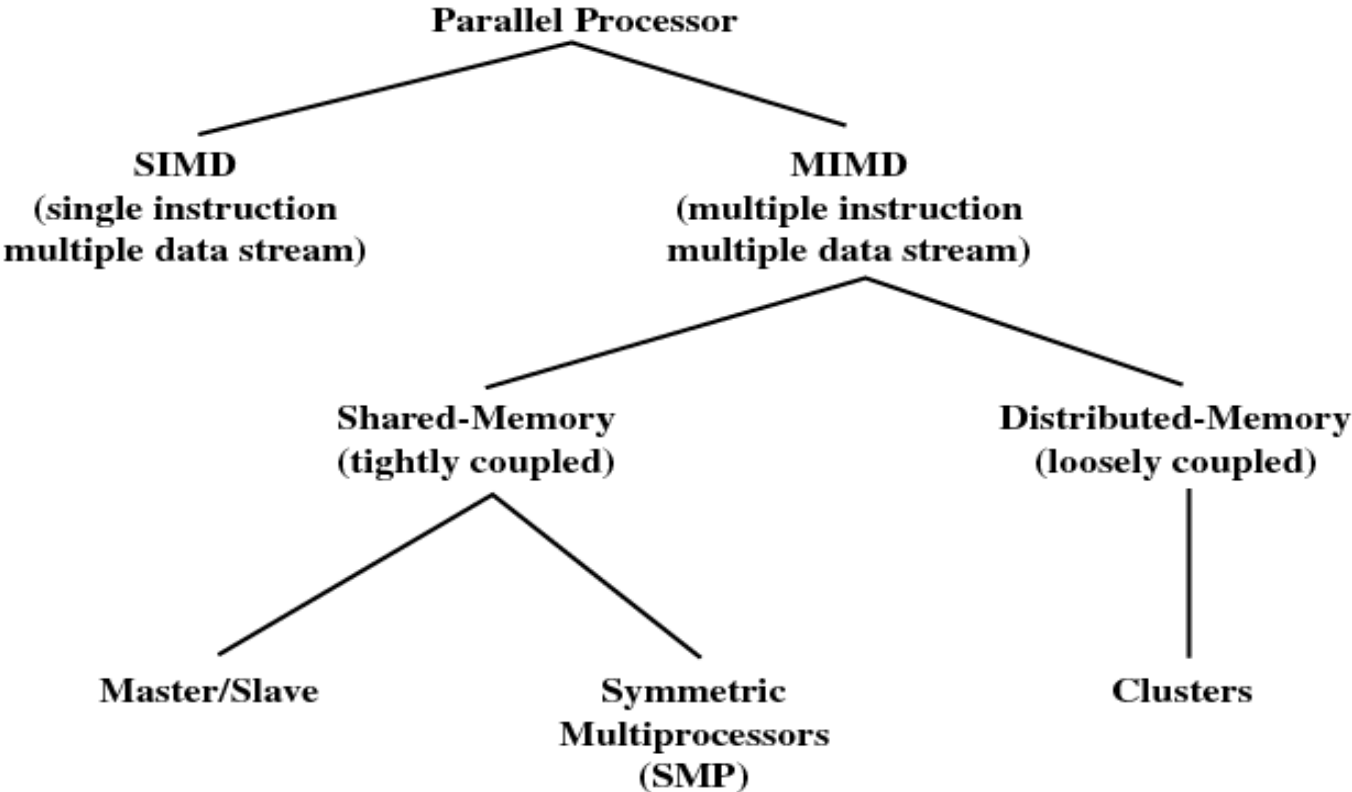
Symmetric Multiprocessing

4 Flynn's taxonomy:

- Single Instruction Single Data (SISD) stream
 - Single processor executes a single instruction stream to operate on data stored in a single memory
- Single Instruction Multiple Data (SIMD) stream
 - Each instruction is executed on a different set of data by the different processors
- Multiple Instruction Multiple Data (MIMD)
 - A set of processors simultaneously execute different instruction sequences on different data sets
- Multiple Instruction Single Data (MISD)
 - A sequence of data is transmitted to a set of processors, each executes a different instruction sequence. Never implemented



Symmetric Multiprocessing





Tightly Coupled vs Loosely Coupled Characteristics

Tightly Coupled	Loosely Coupled
Close proximity to processors	Physically separated processors
High-bandwidth communication (memory bus) via shared memory	Low-bandwidth message-based communication
Single copy of OS	Independent OS



Master / Slave Architecture

- 4 Kernel always run on a particular processor, the master
- 4 The master handles process management and scheduling
- 4 Disadvantages:
 - The failure of the master brings down the whole system
 - The master can become a performance bottleneck

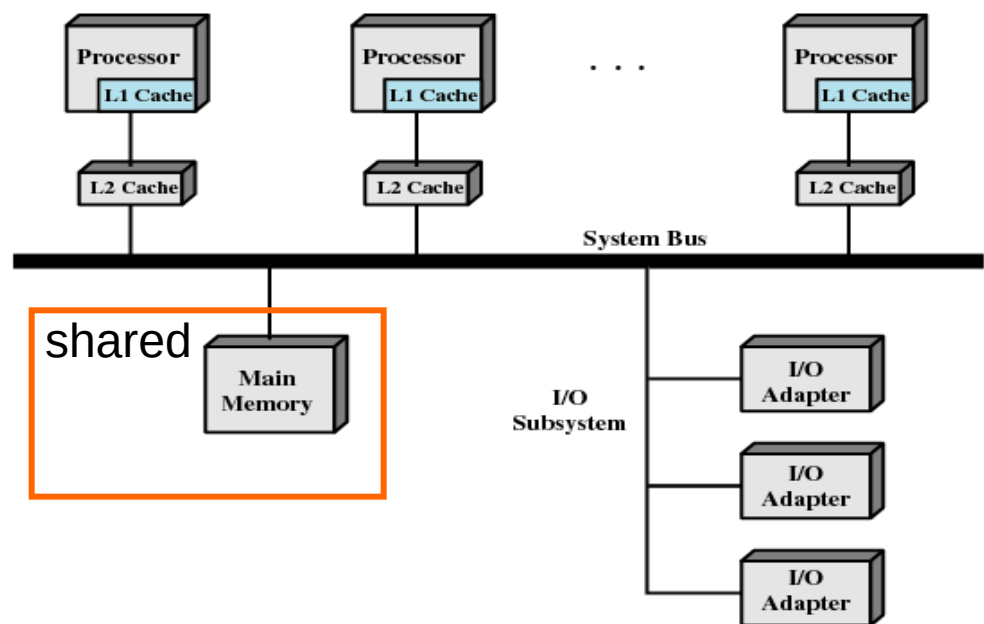


Symmetric Multiprocessor

- 4 Kernel can execute on any processor
- 4 Typically each processor does self-scheduling from the pool of available processes or threads
- 4 Must ensure that 2 processors do not choose the same process



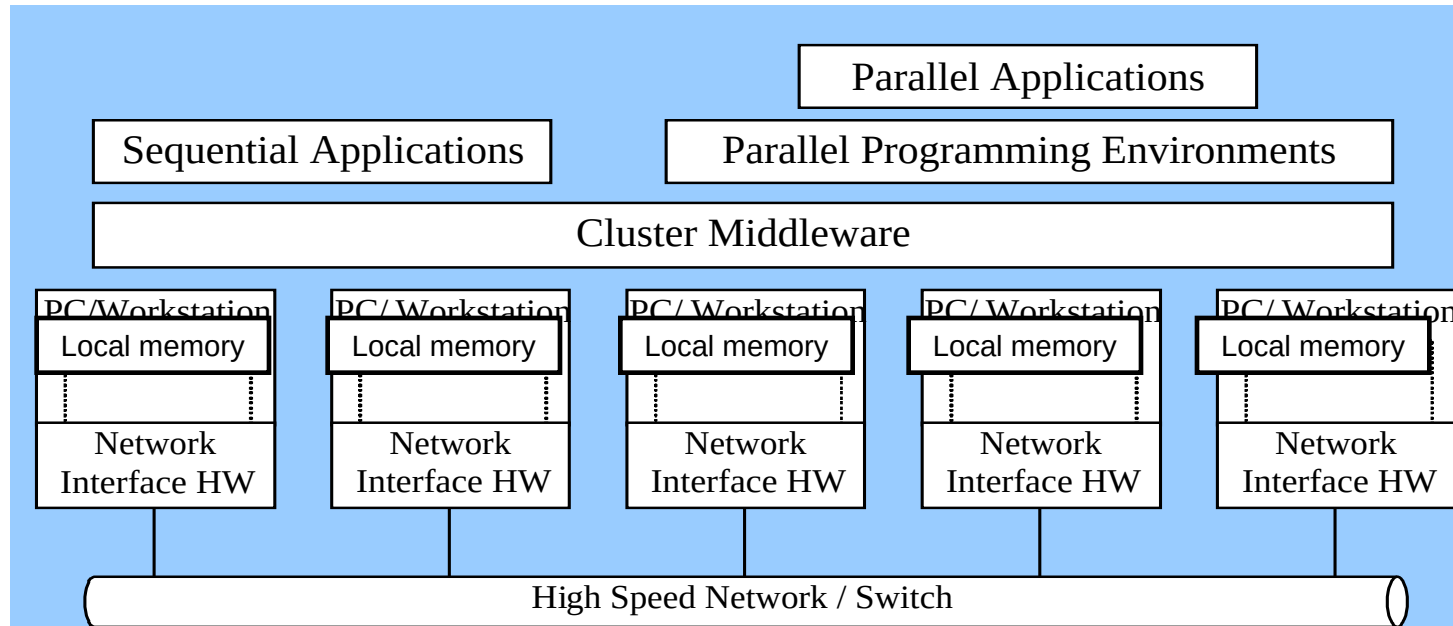
SMP Organization



- 4 A multiprocessor system with shared memory (tightly coupled system).
- 4 Due to symmetric access of all processor to all memory modules -> Symmetrical Multi-Processing (SMP)



Cluster Architecture



- Each processor has its local memory with the address space available only for this processor.
- Processors can exchange data through the interconnection network by means of communication through the **message passing**.



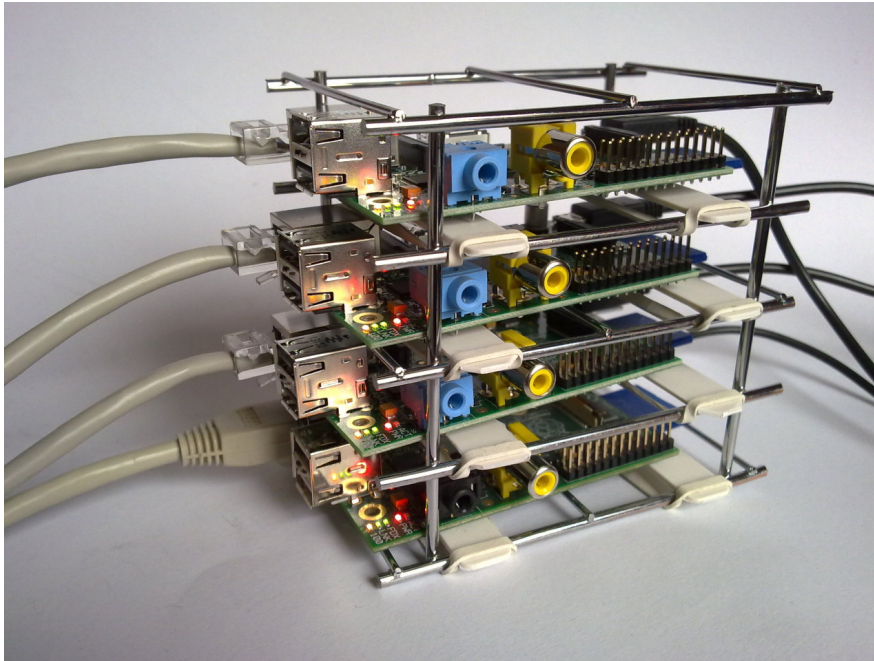
Examples of Cluster



Quartz CTS-1 Cluster



Examples of Cluster



<http://raspberrypiwebserver.com/raspberrypicluster/raspberry-pi-cluster.html>



Multiprocessor OS Design Considerations

- 4 Simultaneous concurrent processes or threads: allow several processors to execute the same kernel code simultaneously, at the same time, avoid deadlock or invalid operations.
- 4 Scheduling: may be performed by any processor, conflicts must be avoided.
- 4 Synchronization: active processes may access to shared address spaces or I/O resources.
- 4 Memory management
- 4 Reliability and fault tolerance: OS must be able to recognize the loss of a processor and restructure management tables accordingly.

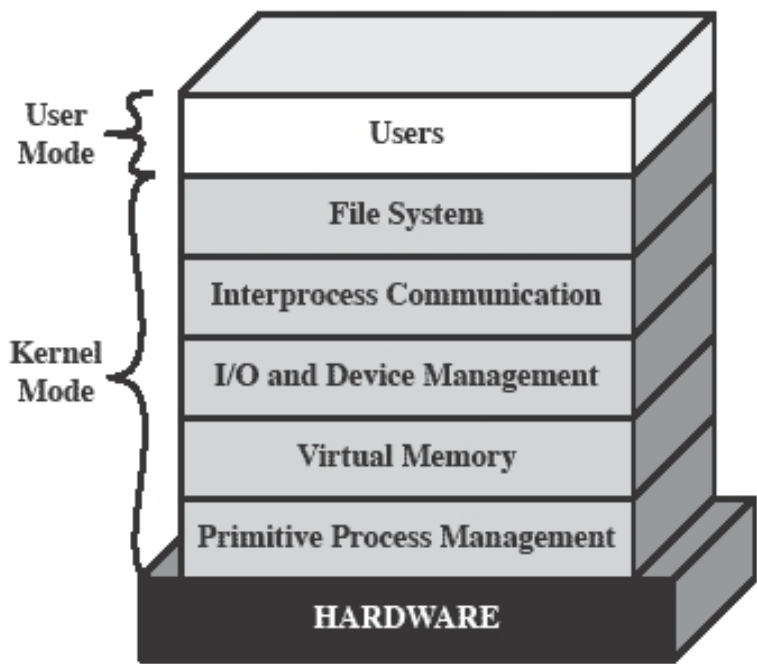


Microkernels

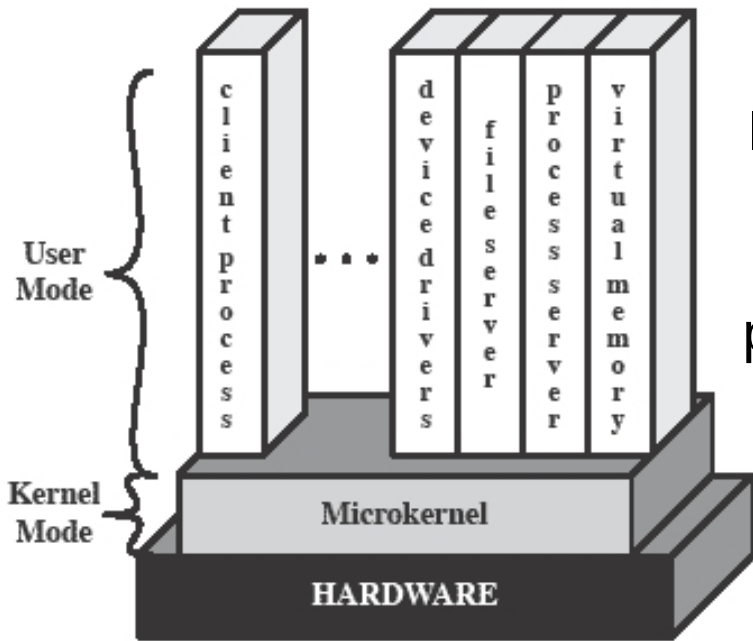
- 4 Small operating system core
- 4 Contains only essential core operating systems functions
- 4 Many services traditionally included in the operating system are now external subsystems
 - Device drivers
 - File systems
 - Virtual memory manager
 - Windowing system
 - Security services



Microkernel Architecture



(a) Layered kernel or monolithic



Implemented as server processes

(b) Microkernel

Figure 4.10 Kernel Architecture



Benefits of Microkernel Organization

- 4 Uniform interface on request made by a process
 - Don't distinguish between kernel-level and user-level services (all services are provided by means of message passing)
- 4 Extensibility
 - Allows the addition of new services without building a new kernel.
- 4 Flexibility
 - New features can be added
 - Existing features can be subtracted
- 4 Portability
 - Changes needed to port the system to a new processor are fewer (changed in the microkernel)



Benefits of Microkernel Organization

4 Reliability

- Modular design
- Small microkernel can be rigorously tested

4 Distributed system support

- Message are sent without knowing what the target machine is (if all processes and services in distributed system have unique identifiers → single system image)

4 Support for Object-oriented operating system

- Components are objects with clearly defined interfaces that can be interconnected to form software



Microkernel Design

- 4 Must include functions that depend directly on the hardware and functions needed to support the servers and applications operating in user mode
 - Low-level memory management
 - Interprocess communication
 - I/O and interrupt management



Microkernel Design

4 Low-level memory management

- Mapping each virtual page to a physical page frame
- Protection of address space, page replacement algorithm and other paging logic are implemented outside the kernel



Microkernel Design

4 Low-level memory management (example)

- When an application references to a page not in main memory → page fault occurs and execution traps to kernel → kernel sends a message to pager → the pager allocate a page frame and load that page → pager sends a resume message to the application.

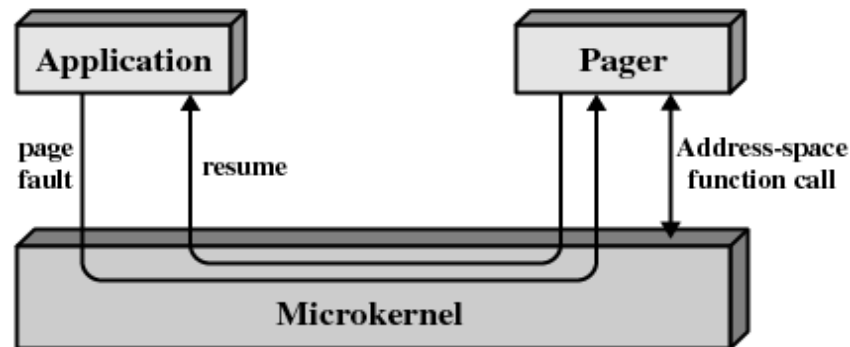


Figure 4.11 Page Fault Processing



Microkernel Design

4 Interprocess communication

- Processes / threads communicate by passing messages

4 I/O and interrupt management

- It is possible to handle interrupts as messages.
- Microkernel recognize interrupts and generates a message for the user-level process that associated with the interrupt → microkernel does not handle interrupts



MINIX 3

4 The system is structured in 4 layers

4	Init	User process	User process	User process	...
3	Process manager	File system	Info system	Network server	...
2	Disk driver	TTY driver	Ethernet driver	...	
1	Kernel task		Clock task	System	



MINIX 3

4 Layer 1

- Kernel mode
- Schedule execution of processes
- Handles IPC (inter process communication)
- Clock task – device driver that interacts with hardware that generates timing signals
- System task provides kernel calls for reading & writing I/O ports, copying data between address spaces



MINIX 3

4 Reincarnation server

- starts / restarts device drivers that are not loaded at the same time as kernel
- starts a fresh copy of driver that fails during operation

** Device drivers typically comprise 70% of the OS code



Monolithic Kernels

4 Case Studies of threads and SMP for monolithic kernels:

- Windows
- Linux



Different Approaches to Processes

4 Differences between different OS's support of processes include

- How processes are named
- Whether threads are provided
- How processes are represented
- How process resources are protected
- What mechanisms are used for inter-process communication and synchronization
- How processes are related to each other

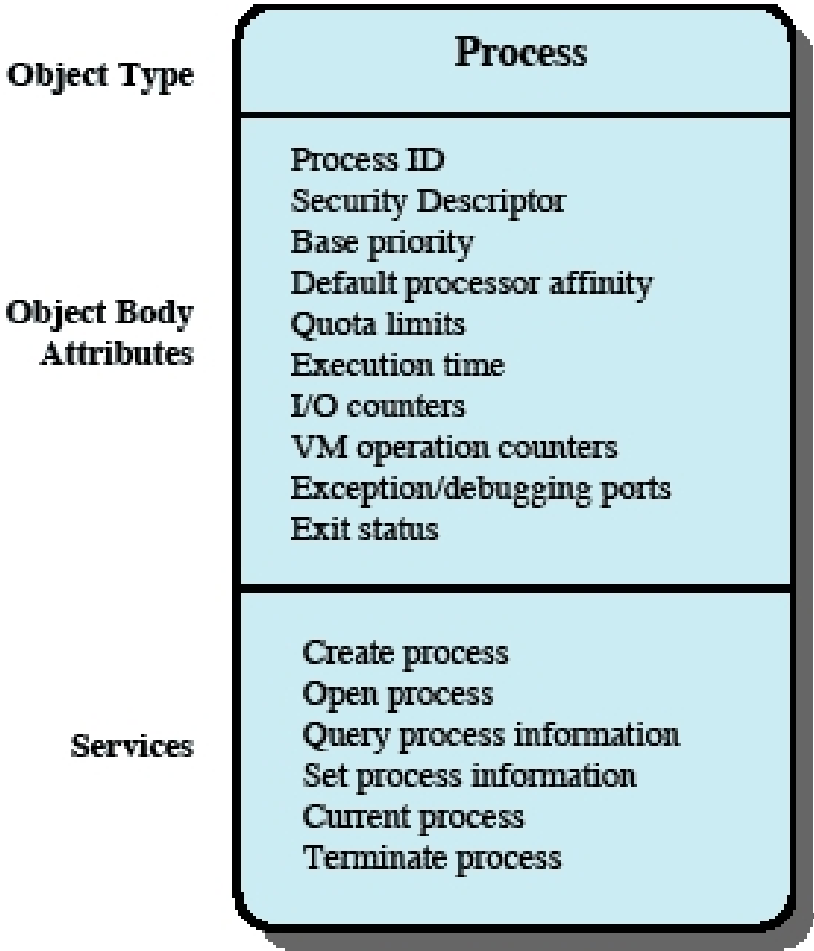


Windows Processes

- 4 Processes and services provided by the Windows Kernel are relatively simple and general purpose
 - Implemented as objects
 - An executable process may contain one or more threads
 - Both processes and thread objects have built-in synchronization capabilities



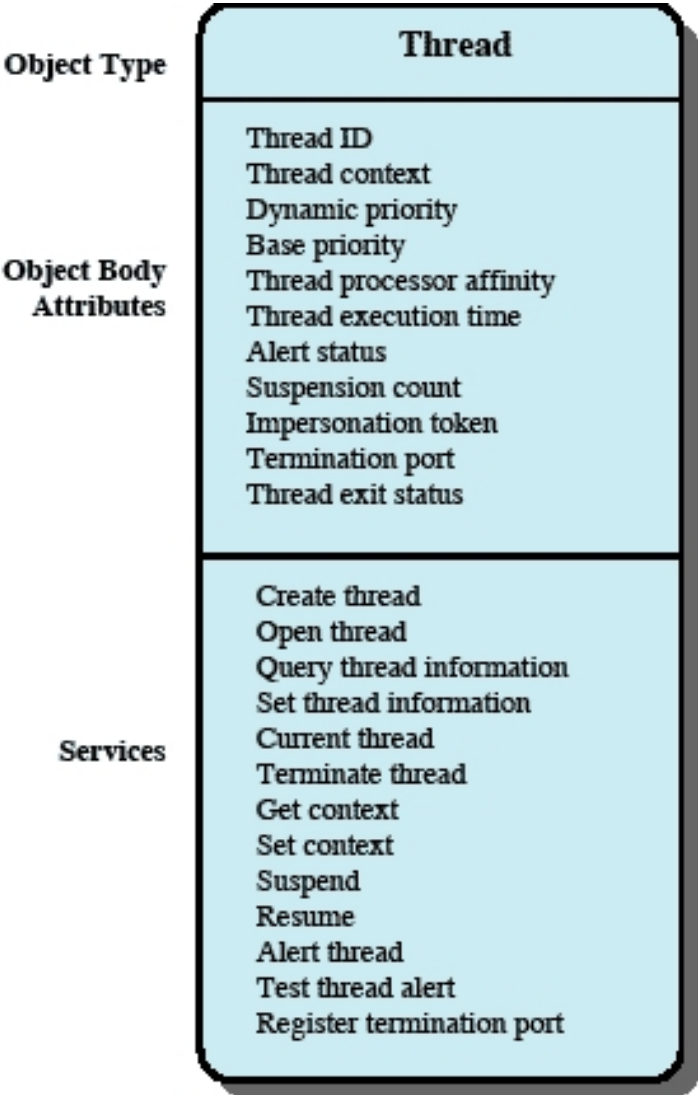
Windows Process Object



(a) Process object



Windows Thread Object



(b) Thread object



Thread States

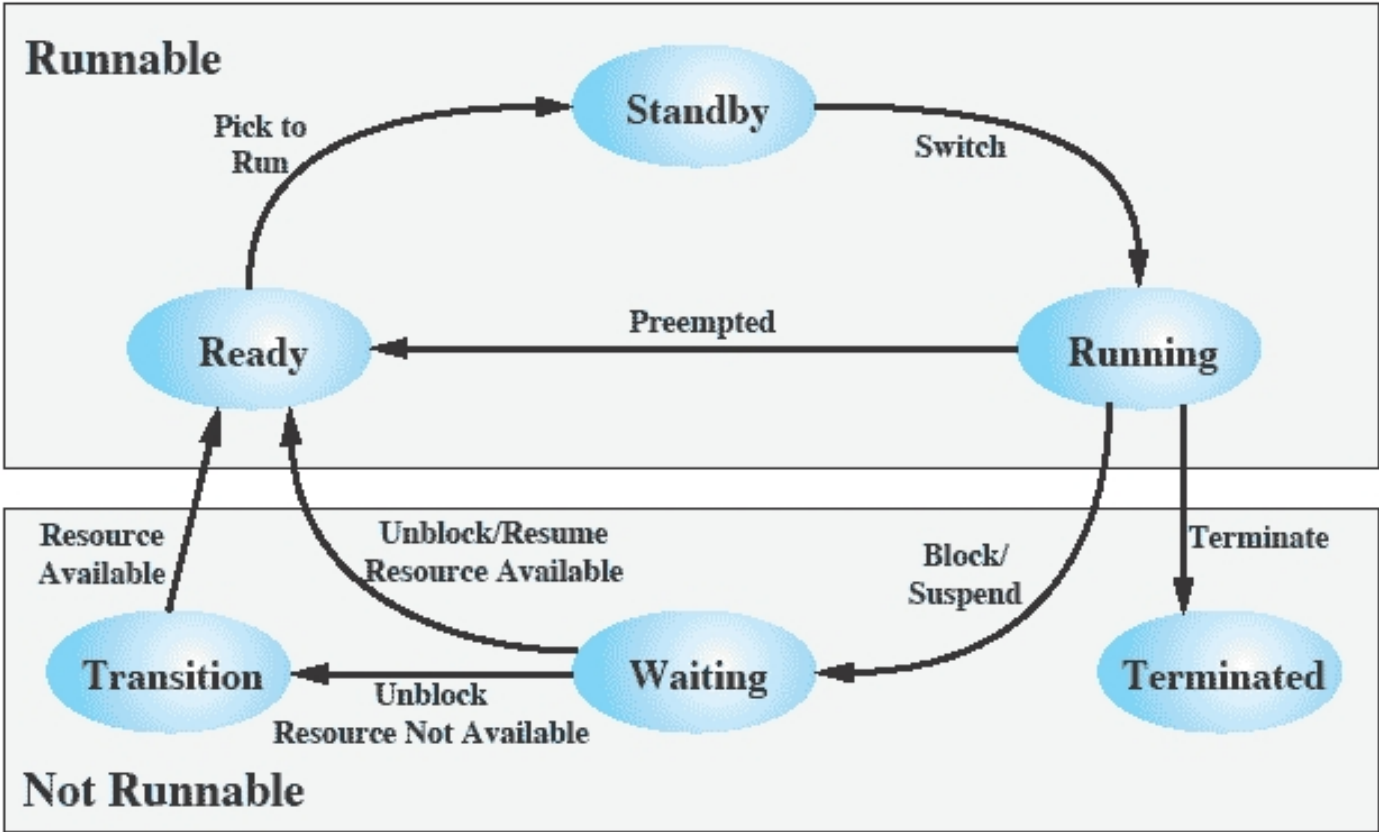


Figure 4.14 Windows Thread States



Windows SMP Support

4 Threads can run on any processor

- But an application can restrict affinity

4 Soft Affinity

- The dispatcher tries to assign a ready thread to the same processor it last ran on.
- This helps reuse data still in that processor's memory caches from the previous execution of the thread.

4 Hard Affinity

- An application restricts threads to certain processor



Linux Tasks

- 4 Task (which can be process or thread*) in Linux is represented by a `task_struct` data structure
- 4 This contains a number of categories including:
 - State
 - Scheduling information
 - Identifiers
 - Interprocess communication
 - And others

* Linux does not really differentiate process and thread from kernel point of view. The kernel just views thread as another process with shared resources.



Linux Process/Thread Model

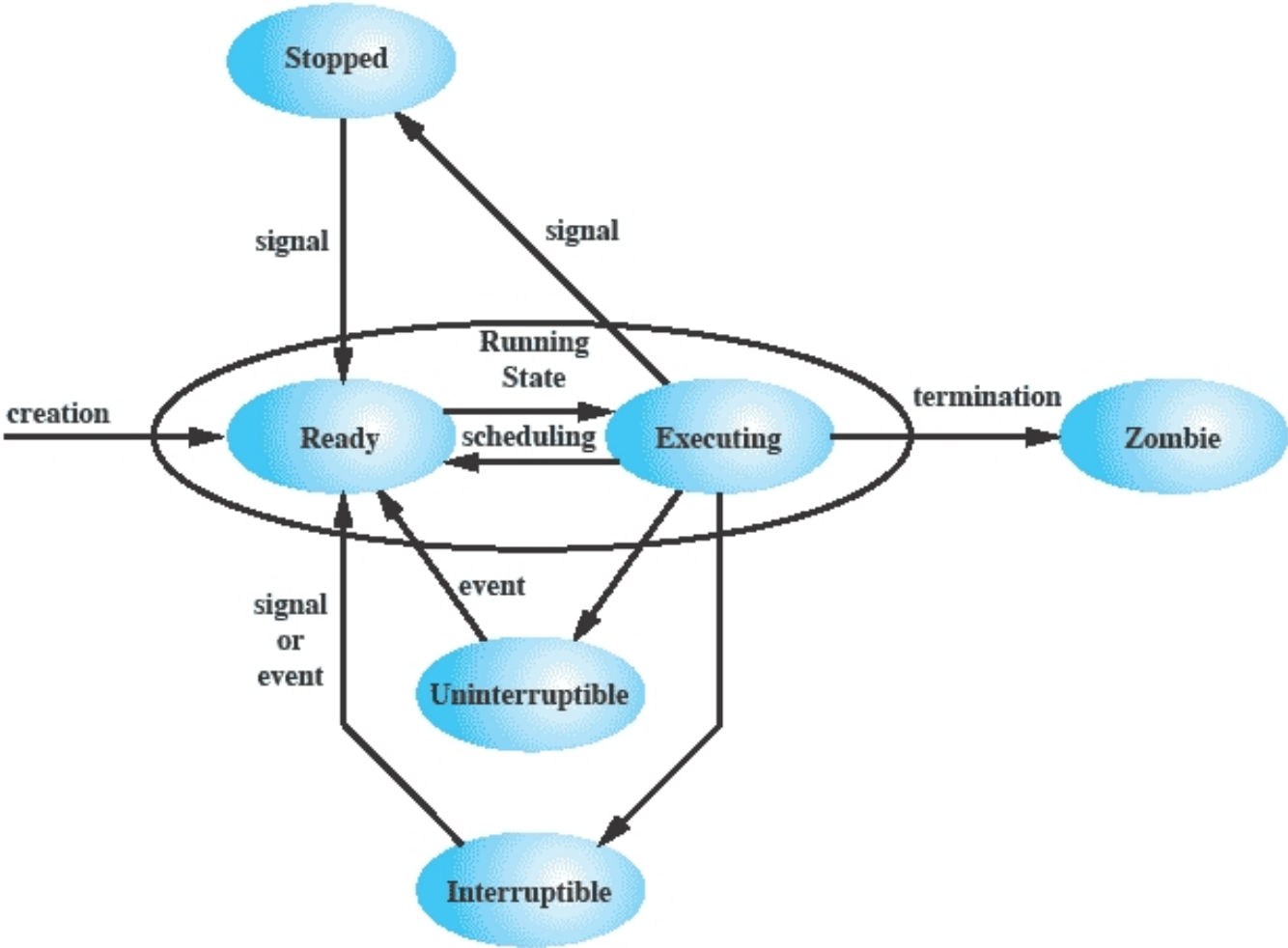


Figure 4.18 Linux Process/Thread Model